

Amrutam Super App

Complete system design, feature flows, architecture decisions, and operational behaviour for the Ayurvedic Consultation · Shop · Health Records mobile application.

Document type	Senior Software Engineering — System & Feature Guide
Platform	React Native 0.87.1 · React 19 · TypeScript
Audience	Engineers, reviewers, product stakeholders
Version	1.0 · August 2026
Repository	AssignmentApp (feature-oriented Clean Architecture)
Data scale	5,000 doctors · 20,000 products · 10,000 health records

Table of Contents

- 1. 1. Executive Summary
- 2. 2. Product Scope & Modules
- 3. 3. Technology Stack
- 4. 4. System Architecture
- 5. 5. Application Bootstrap & Navigation
- 6. 6. Authentication & Session
- 7. 7. Consultation Module (End-to-End)
- 8. 8. Shop Module (End-to-End)
- 9. 9. Health Records Module (End-to-End)
- 10. 10. Settings & Reliability Simulation
- 11. 11. Offline-First Strategy
- 12. 12. Performance Strategy
- 13. 13. Mock API & Data Generation
- 14. 14. Design System & UX Patterns
- 15. 15. Testing Strategy
- 16. 16. Appendix — Key Paths & Glossary

1. Executive Summary

Amrutam Super App is a production-style React Native application that unifies three healthcare commerce experiences: Ayurvedic doctor consultation booking, an herbal product shop, and a patient health-record timeline. The assignment emphasises **senior engineering practices** rather than a thin UI demo:

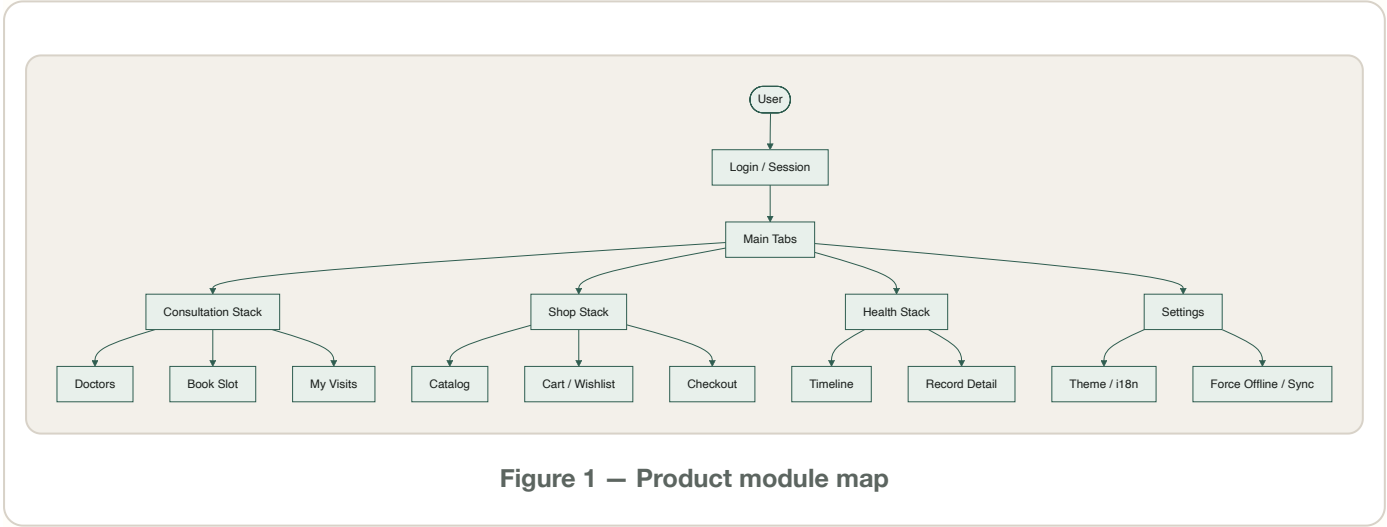
- Feature-oriented Clean Architecture (domain / data / presentation)
- Offline-first storage, API response cache, and booking sync queue
- Virtualized lists over large mocked datasets (5k / 20k / 10k)
- Reliability simulation (slow network, failures, session expiry)
- Deterministic seeded generators — no bulky fixture JSON

Design principle: Screens talk to repositories via TanStack Query; repositories talk to `apiClient`; mocks live behind the API boundary. UI never imports generators directly.

2. Product Scope & Modules

Module	User value	Primary screens
Consultation	Find doctors, book/cancel slots, manage visits	Doctor List, Detail, Booking Confirm, Upcoming, Booking Detail
Shop	Browse products, cart, wishlist, checkout	Product List, Detail, Cart, Wishlist, Checkout
Health Records	Personal timeline of clinical documents	Patient Timeline, Record Detail
Settings	Theme, language, offline demo, reliability tools	Settings
Auth	Session gate for the main experience	Login

2.1 High-level product map



3. Technology Stack

Layer	Choice	Notes
Mobile framework	React Native 0.87.1	CLI app (iOS/Android)
UI runtime	React 19.2	Hooks, Suspense for lazy screens
Language	TypeScript	Strict feature typing
Navigation	React Navigation v7	Stack + bottom tabs + deep links
Server state	TanStack Query v5	Infinite queries, keepPreviousData
Client state	Zustand	Session, cart, wishlist, sync queue
Persistence	AsyncStorage	Prefixed keys @amrutam:
Lists	FlashList	Virtualized recycling
Network awareness	NetInfo	online / offline / reconnecting
i18n	i18next	English + Hindi
Tests	Jest + RTL	Domain, pagination, offline sync

4. System Architecture

4.1 Layered feature architecture

Each feature (consultation, shop, health-records) is split into *presentation*, *domain*, and *data*. Shared infrastructure lives under `src/core` ; reusable UI under `src/design-system` .

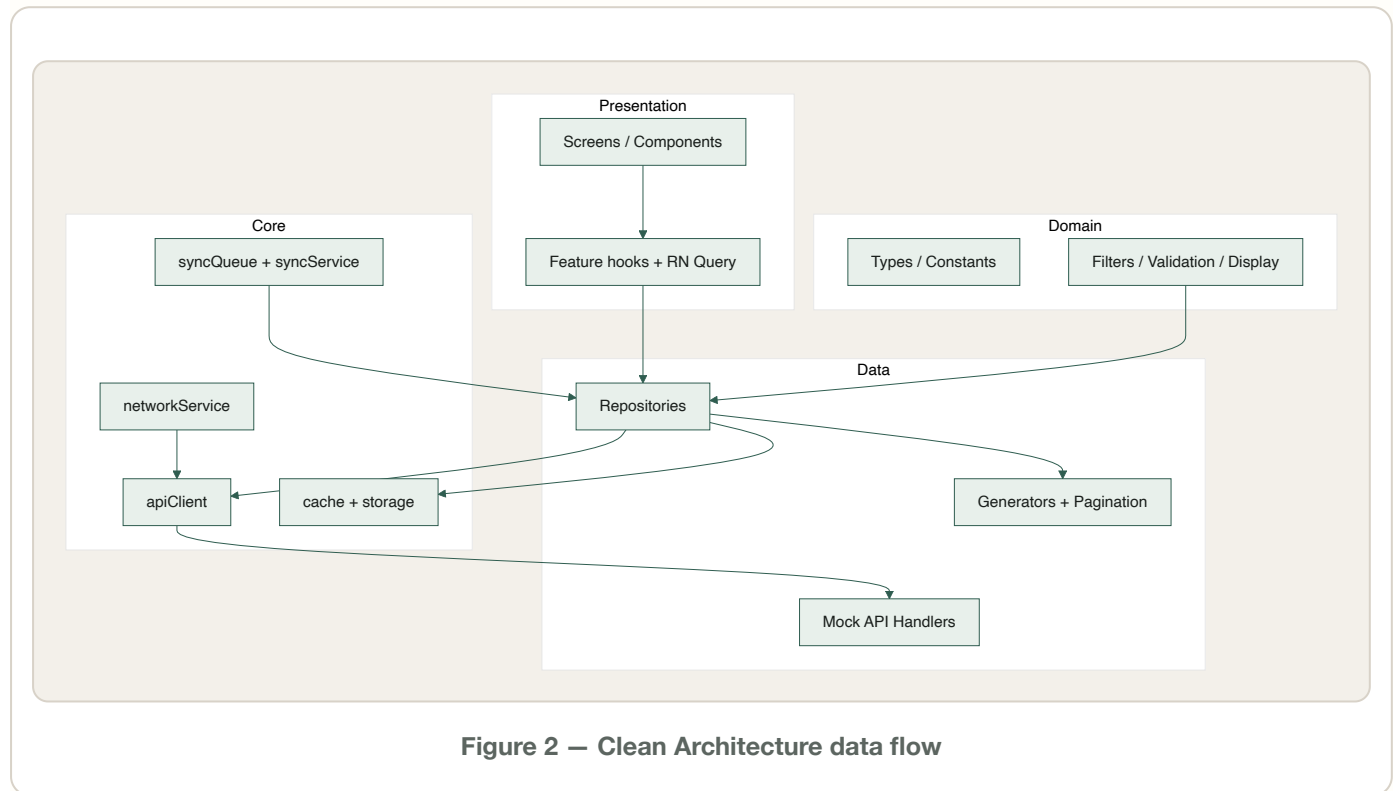


Figure 2 — Clean Architecture data flow

4.2 Source tree (summary)

```
src/  
  app/           navigation, providers, bootstrap  
  core/          api, auth, cache, network, sync, i18n, errors, logging  
  design-system/ theme tokens + UI primitives  
  features/      auth | consultation | shop | health-records | settings  
  shared/        FilterChipBar, FilterPanel, shared hooks  
  testing/       Jest setup
```

5. Application Bootstrap & Navigation

5.1 Cold-start sequence

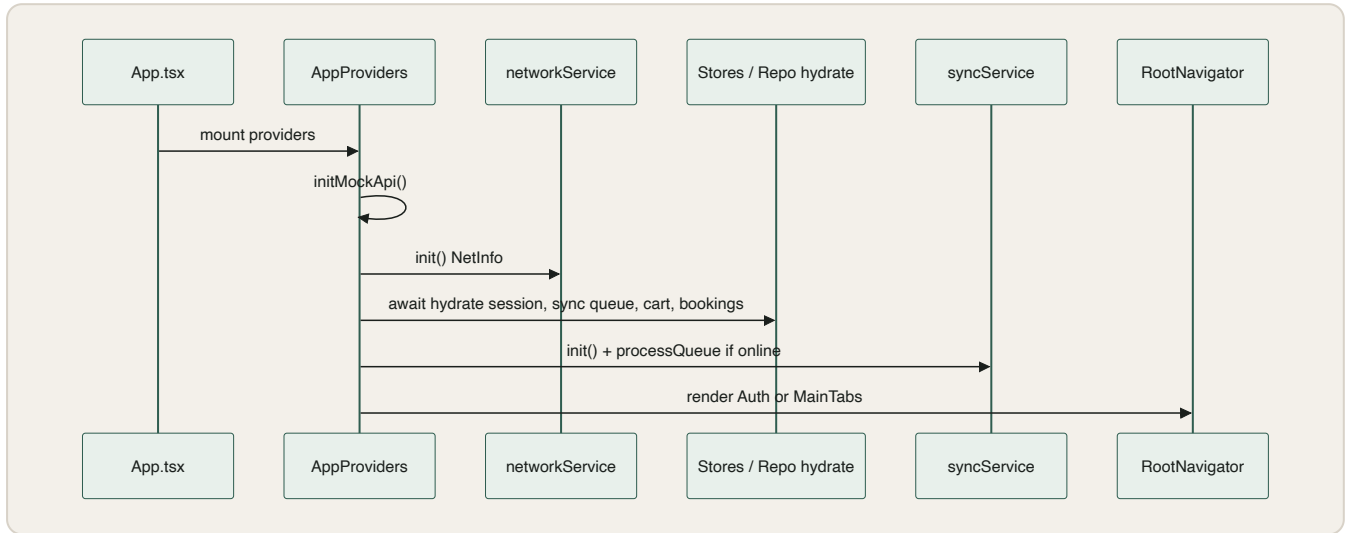


Figure 3 — Bootstrap sequence (hydrate before sync)

5.2 Navigation structure

- **Root stack:** `Auth` | `MainTabs`
- **Tabs:** `Consultation` · `Shop` · `Health` · `Settings`
- **Lazy loading:** detail / secondary screens via `lazyScreen()` (`React.lazy` + `Suspense`)
- **Deep links:** `amrutam://doctor/:id`, `amrutam://product/:id`, `cart`, `health`, `login`, etc.

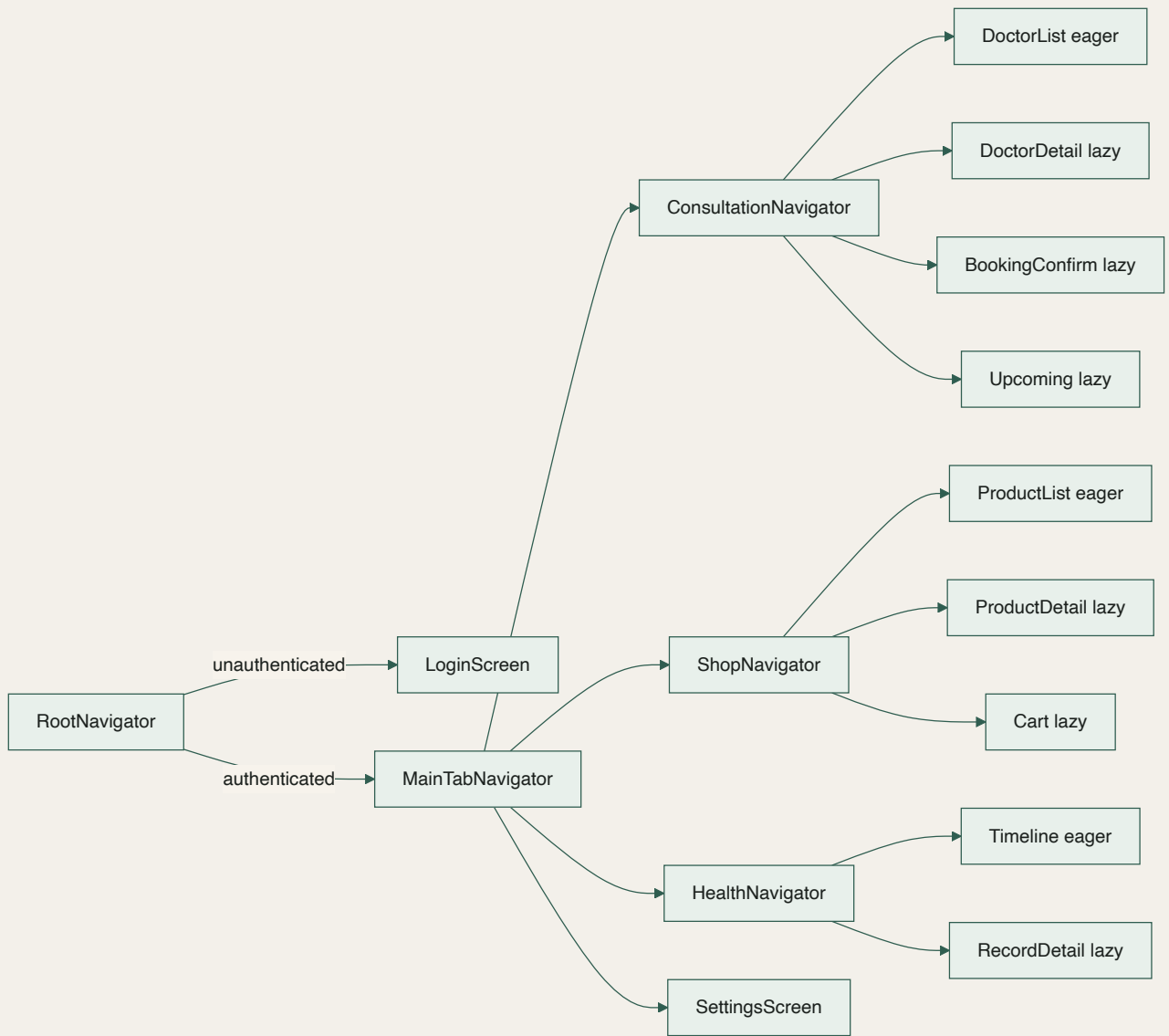


Figure 4 — Navigator graph

6. Authentication & Session

Demo authentication accepts any email/password. A successful login stores `mock-session-token` in Zustand + AsyncStorage. The app may boot already authenticated for smoother local demos. Session expiry (Settings or API simulation) clears the token and returns the user to Login via `SessionGuard`.

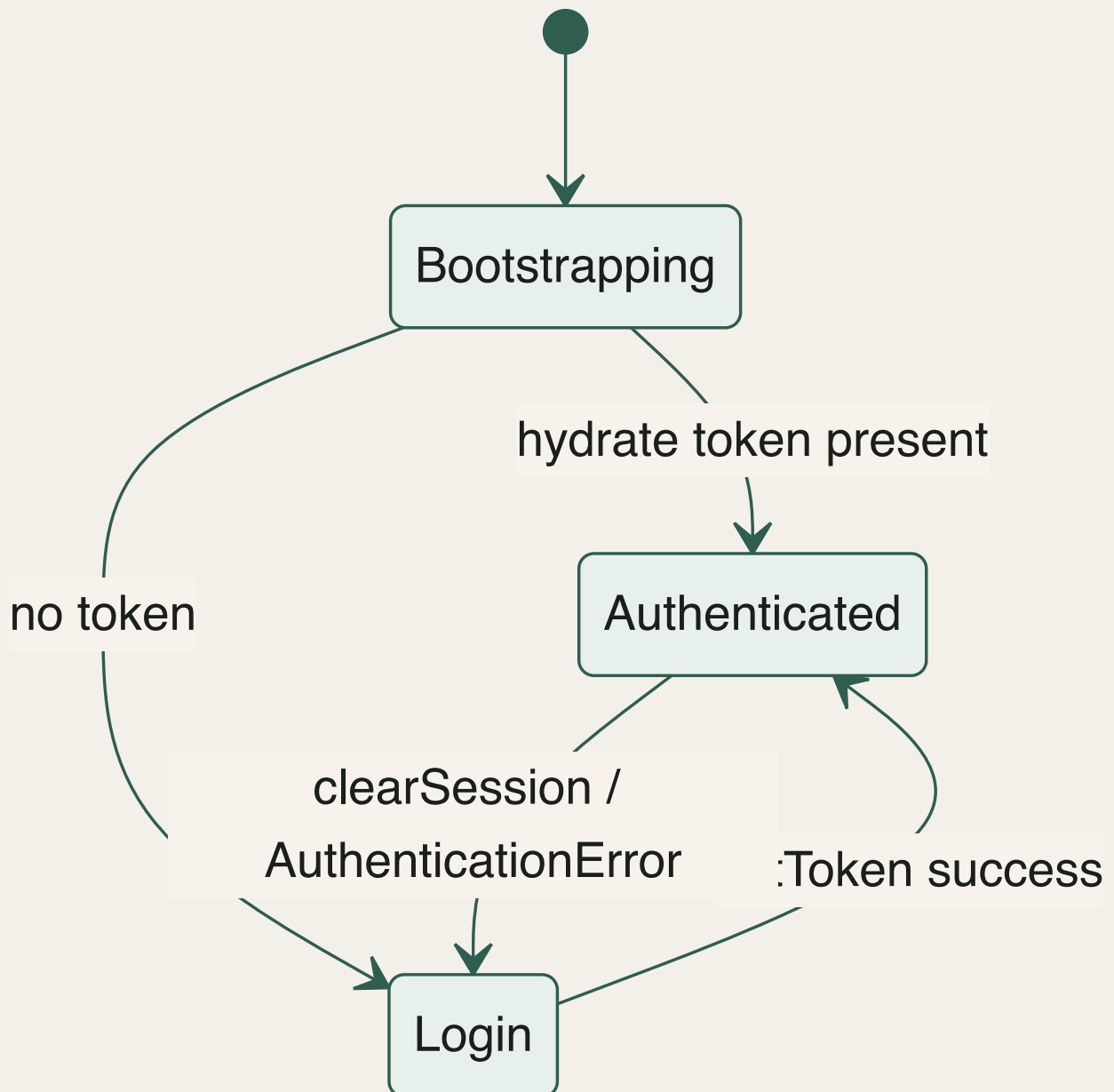


Figure 5 — Session state machine

Concern	Implementation
Store	<code>src/core/auth/sessionStore.ts</code>
Gate	<code>RootNavigator</code> + <code>SessionGuard</code>
API auth	<code>apiClient</code> attaches token; 401 → <code>clearSession</code>

7. Consultation Module (End-to-End)

7.1 Capabilities

- Browse **5,000** doctors with search and multi-filters
- Doctor detail with generated availability slots (7 days)
- Book with client-generated **idempotency key**
- Upcoming visits + cancel (2-hour window rule)
- Offline book/cancel queued and auto-synced on reconnect

7.2 Booking happy path

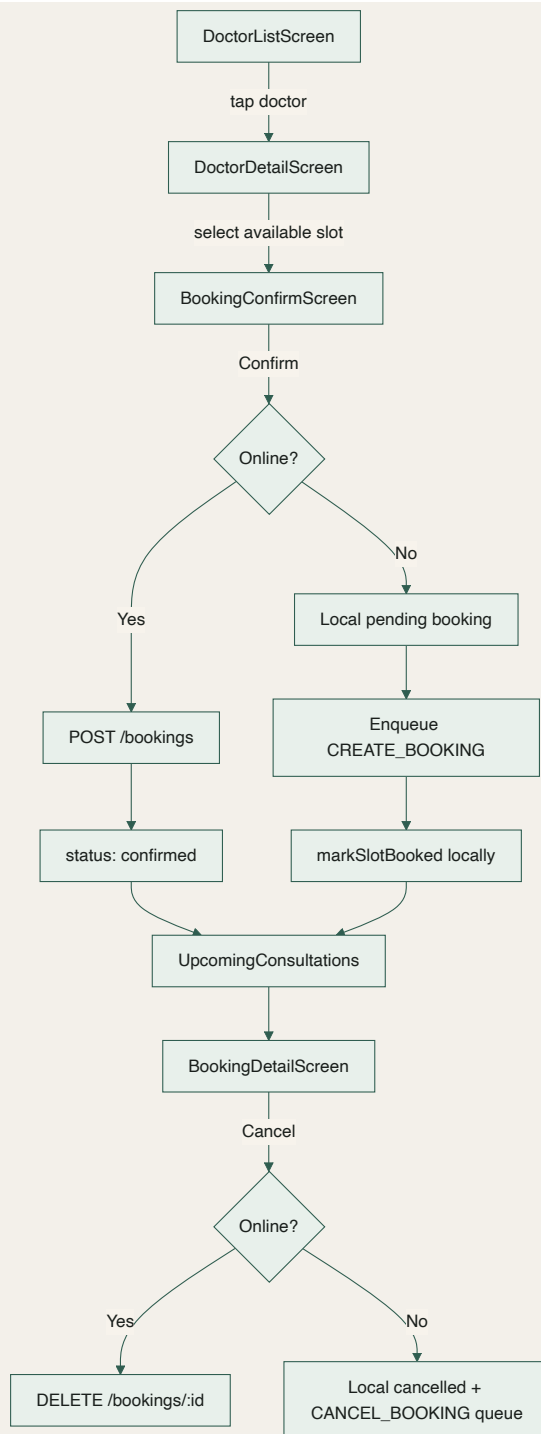


Figure 6 — Consultation booking & cancel flow

7.3 Offline sync pipeline

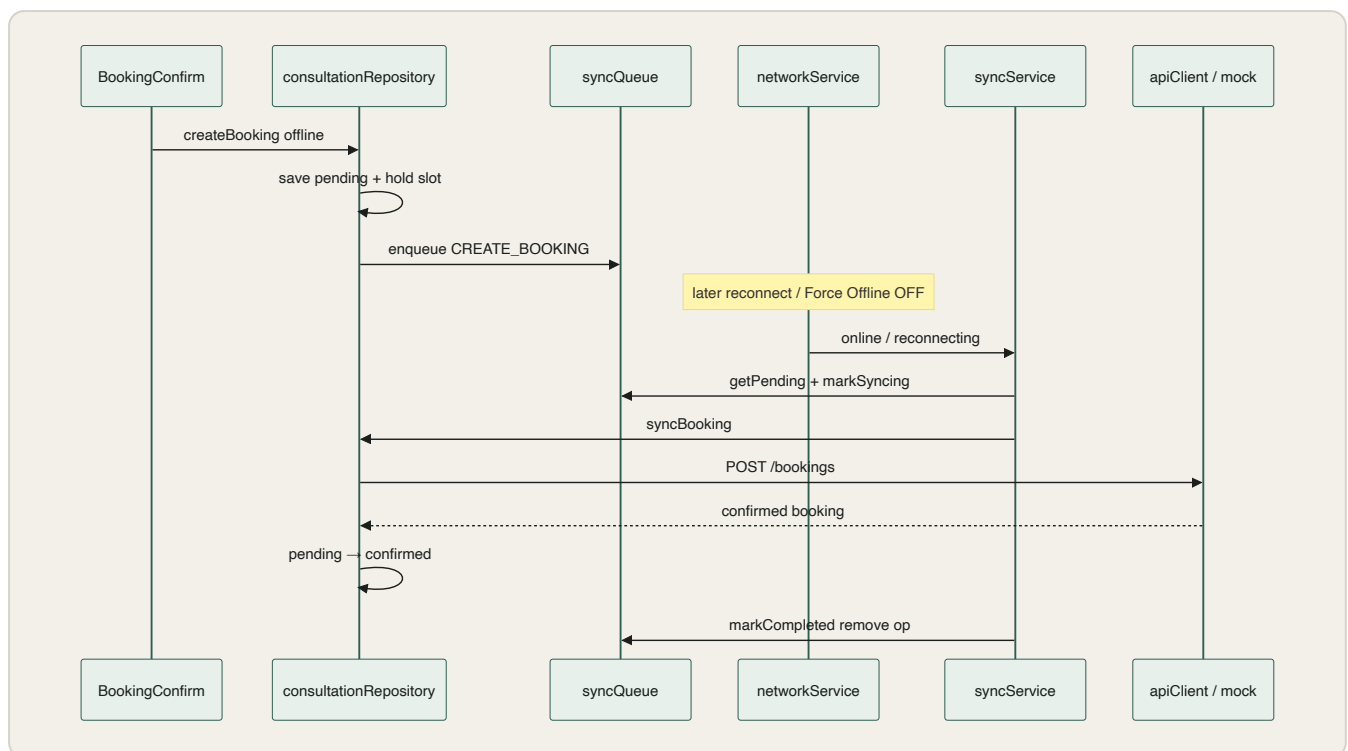


Figure 7 — Offline booking sync sequence

7.4 Filters & pagination

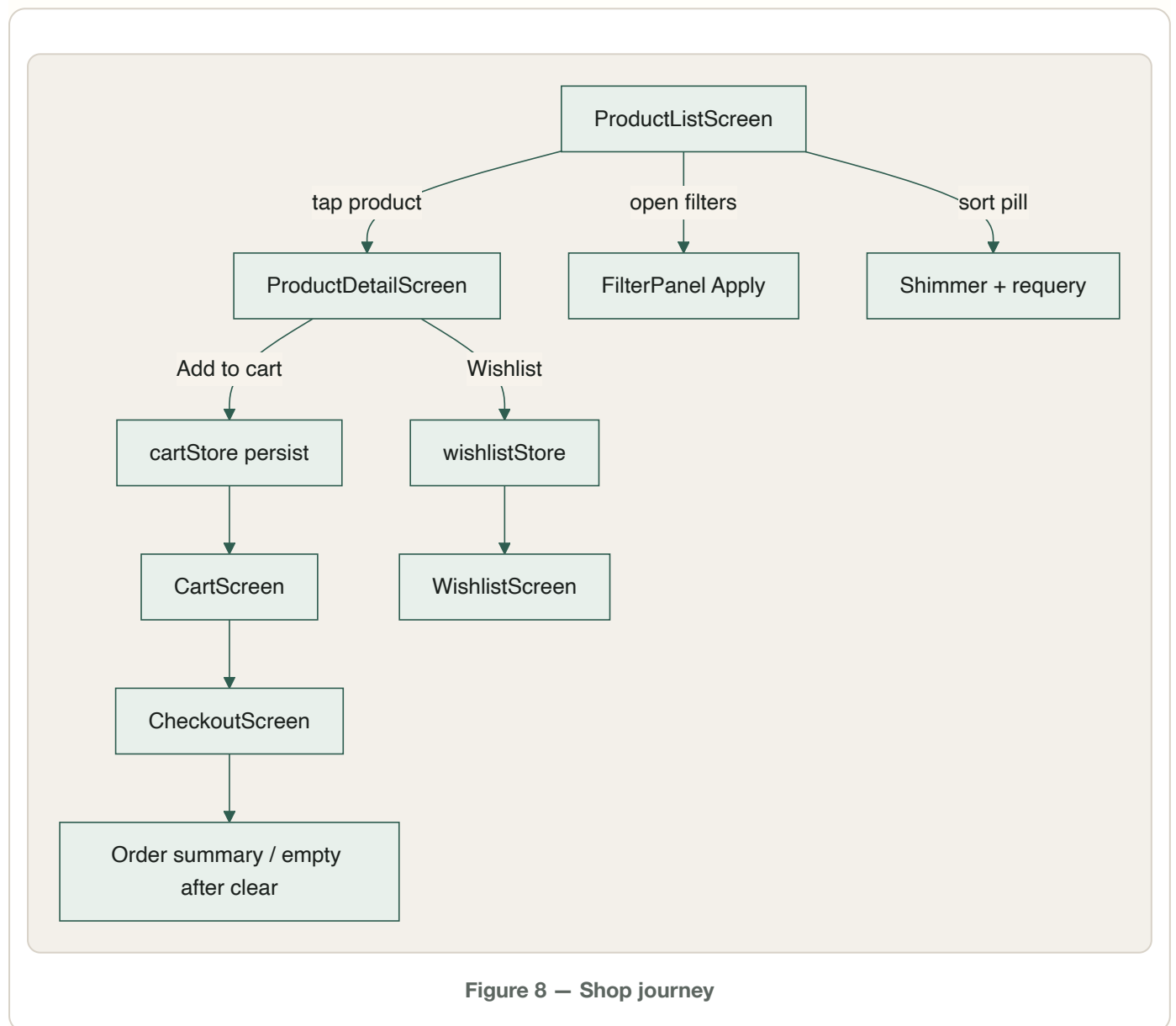
List uses `useInfiniteQuery` (page size 20). Data layer `getDoctorsPage` streams matches without materialising the full filtered 5k array for each page. Filters include specialisation, experience, rating, fee, language, and availability.

8. Shop Module (End-to-End)

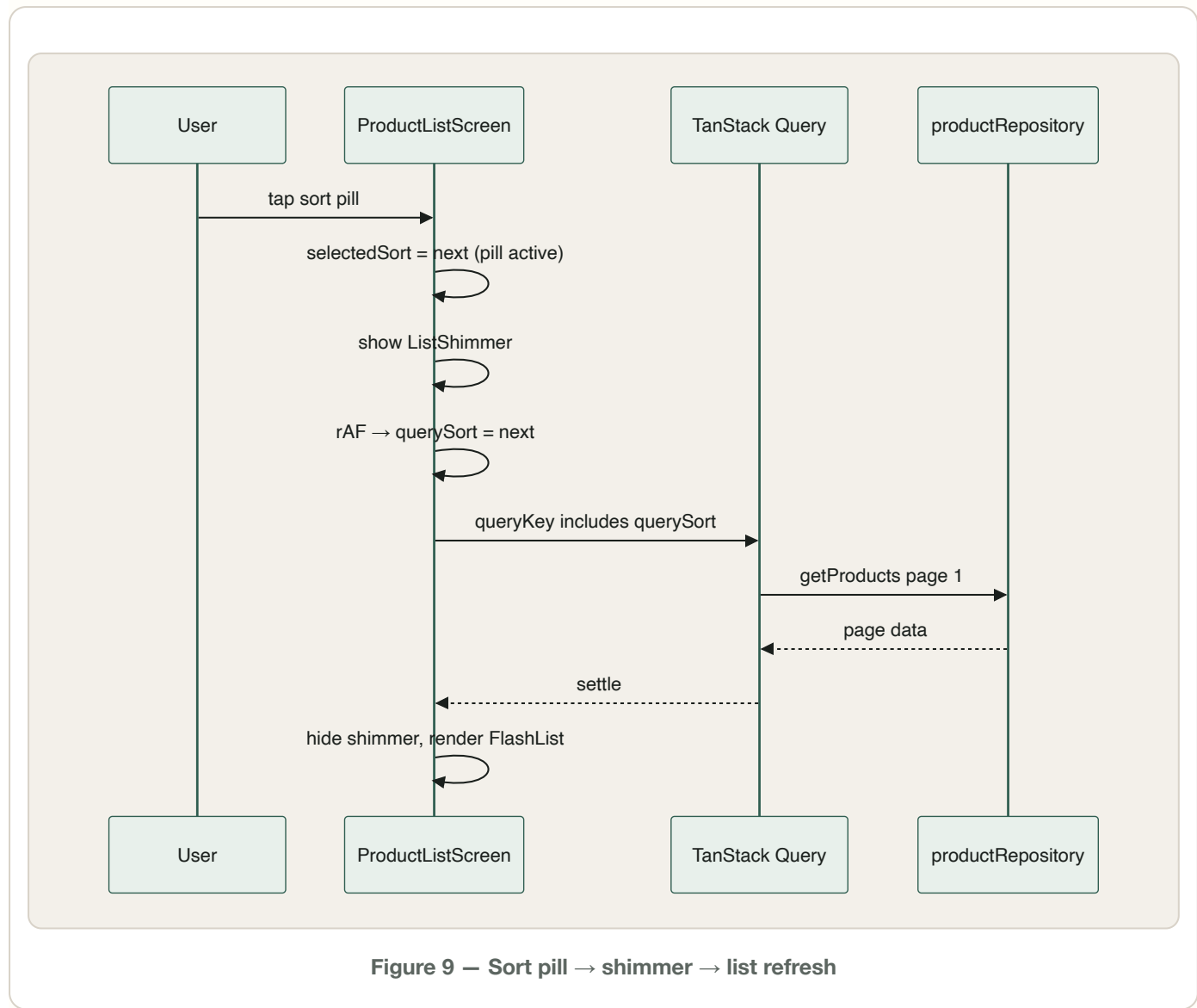
8.1 Capabilities

- Catalog of **20,000** products with category/brand/price/rating/tag filters
- Sort pills: popularity, price asc/desc, rating, newest — immediate selection + shimmer refresh
- Persistent offline cart; wishlist via Zustand persist
- Stock awareness = catalog stock – cart quantity

8.2 Shop user journey



8.3 Sort refresh behaviour



8.4 Cart persistence

Cart mutations write to AsyncStorage (`@amrutam:cart`). On bootstrap, `cartStore.hydrate()` restores items so the shop remains usable offline. Checkout is a local demo summary (no payment gateway).

9. Health Records Module (End-to-End)

9.1 Capabilities

- **10,000** records across five types: Lab Report, Prescription, Consultation, Vaccination, Allergy
- Timeline grouped by **Year** → **Month**, newest first from 2026
- Search, type chips, Year/Month filter panel, tags
- Attachment thumbnails with large preview modal (image/PDF placeholders)

9.2 Timeline rendering model

Instead of nested SectionList, the screen flattens groups into FlashList rows of kinds `year` | `month` | `record` with `getItemType` for efficient recycling.

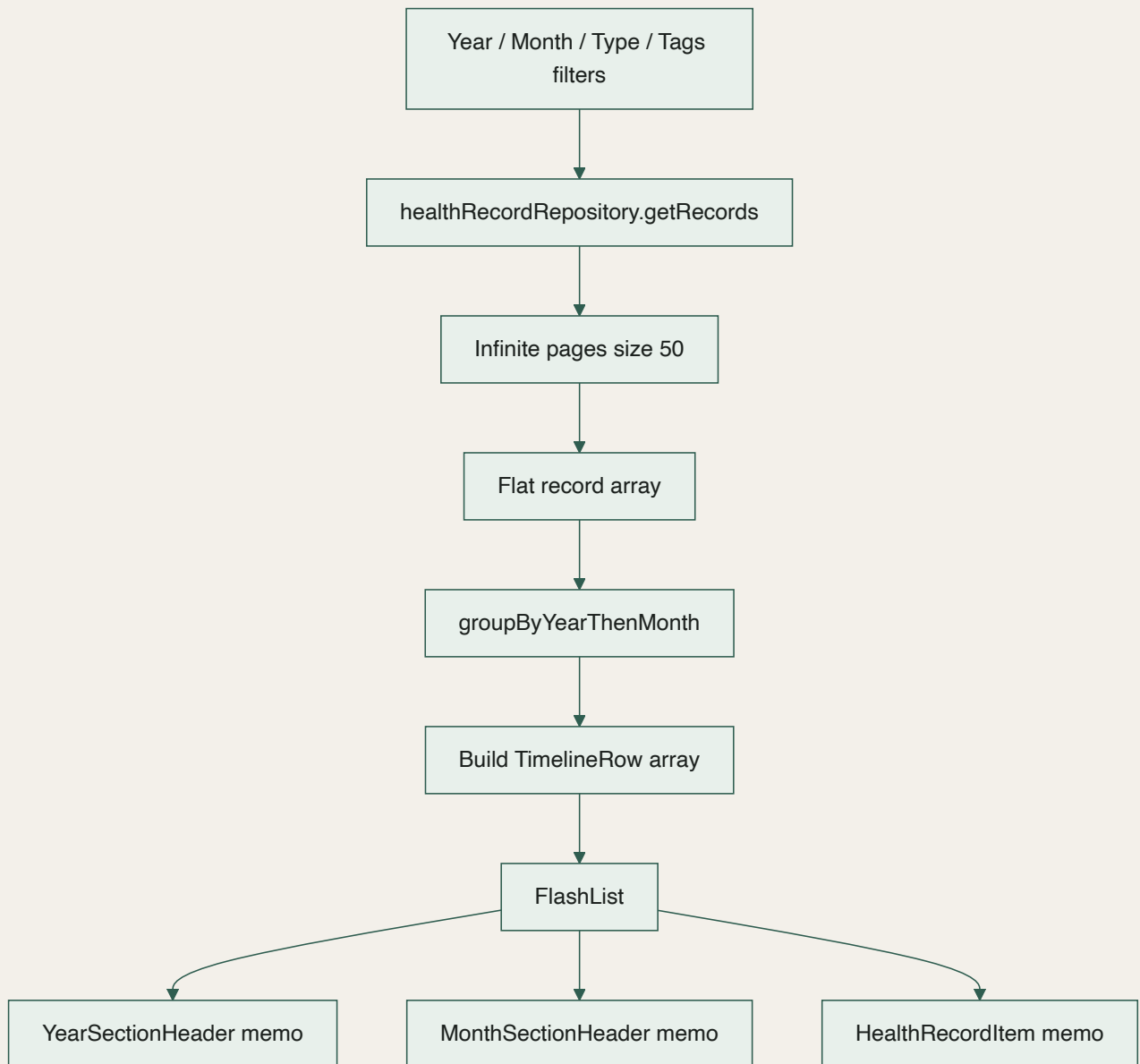
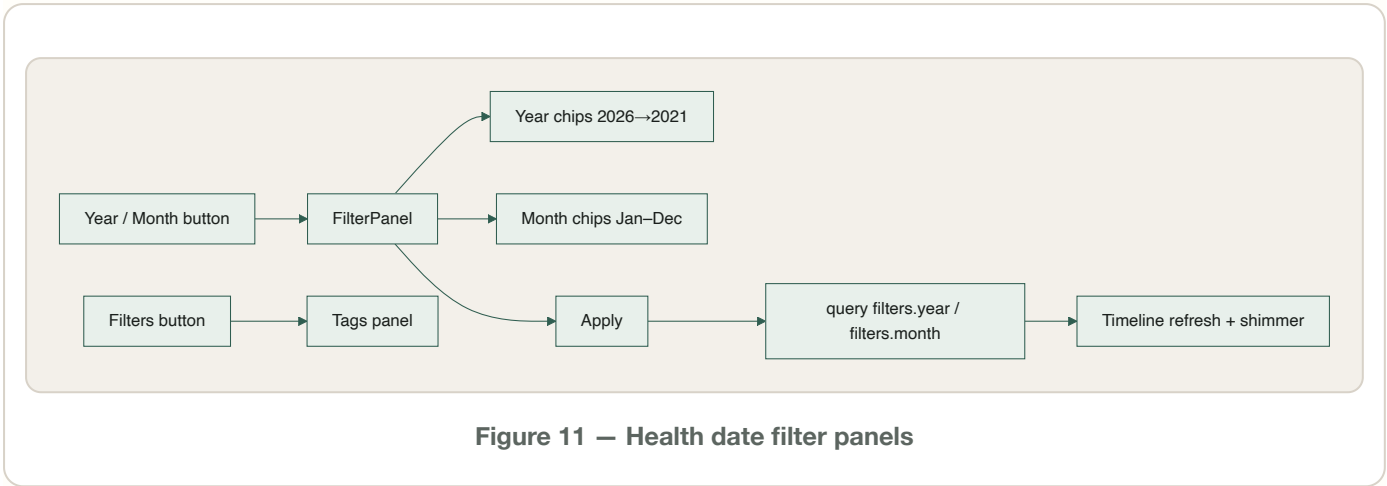


Figure 10 — Health timeline pipeline

9.3 Month / year filter UX



10. Settings & Reliability Simulation

Control	Effect
Force Offline	Marks network offline; APIs fail; cache/queue paths used
Sync queued bookings	Manual <code>syncService.processQueue()</code>
Theme	light / dark / system
Language	English / Hindi
Simulate Session Expiry	Clears session → Login
Random Failures / Slow Network / Bad JSON / Partial	apiClient failure simulation (dev)
Performance Benchmark	First-page fetch timings for 5k/20k/10k

11. Offline-First Strategy

11.1 Pillars

1. **Cached API responses** — TTL cache for doctors, products, health, slots; fresh-or-stale fallback
2. **Offline cart** — persisted cart hydrate on bootstrap
3. **Offline bookings** — pending local booking + sync queue + local slot hold
4. **Automatic sync** — on reconnect (and Settings manual sync)

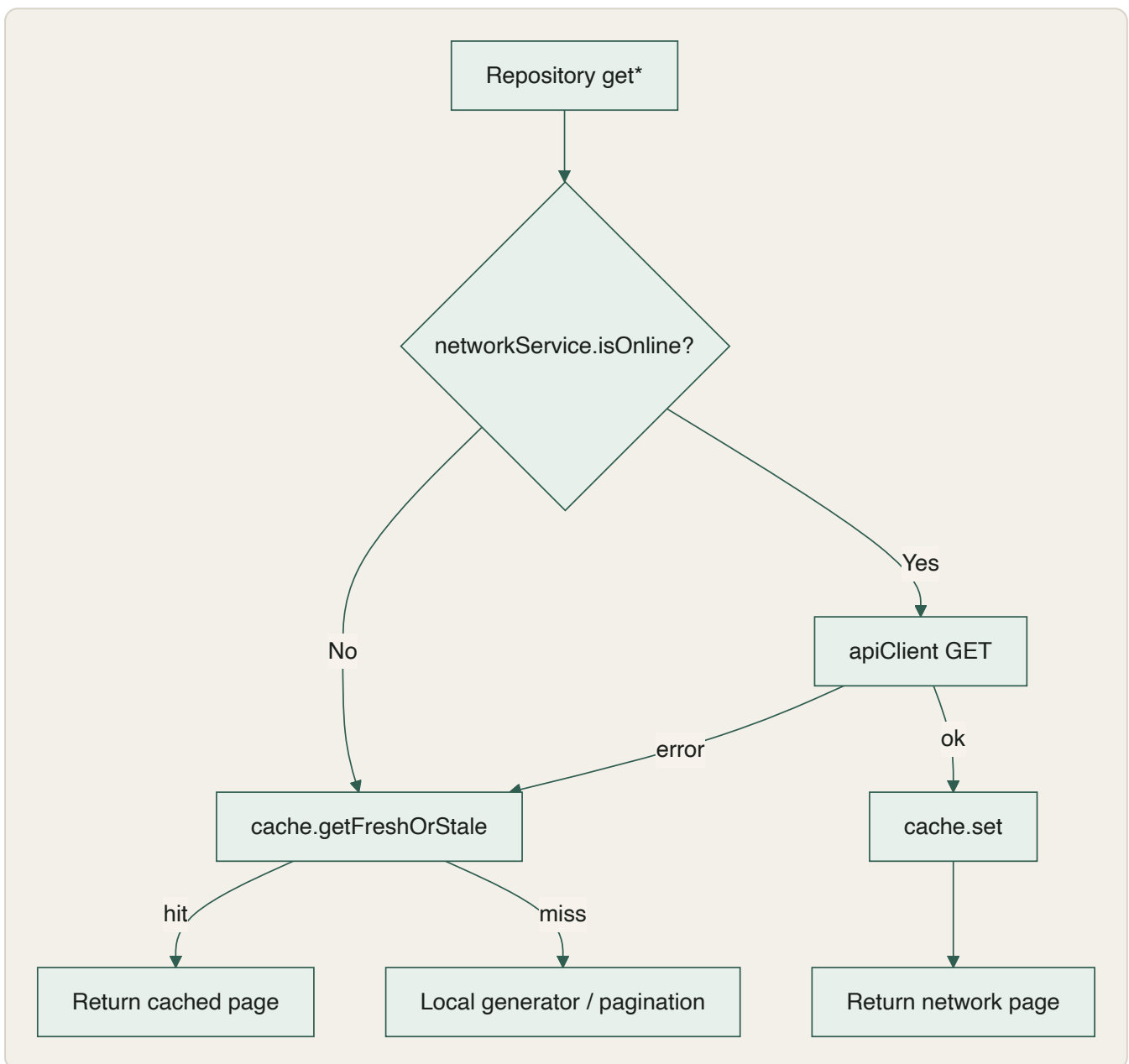
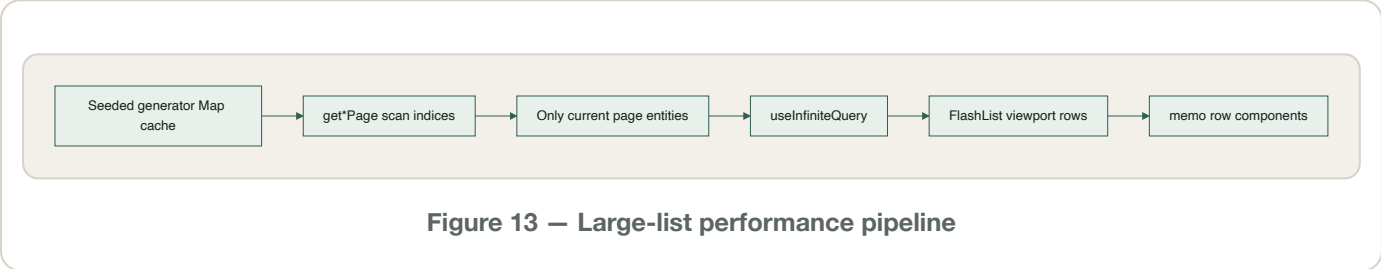


Figure 12 — Read path: network → cache → local

When offline, `apiClient` rejects *all* methods (including GET) so repositories demonstrably serve cache/local data rather than silently hitting the mock.

12. Performance Strategy

Technique	How demonstrated
Virtualized rendering	FlashList on doctors, products, health timeline
Memoization	<code>React.memo</code> list rows & section headers; stable <code>renderItem</code>
Efficient state updates	Debounced search 300ms; deferred filters; narrow Zustand selectors
Lazy loading	Infinite query pages + <code>lazyScreen</code> for secondary routes
Streaming pagination	doctor/product/health pagination fills a page without full filtered materialisation



13. Mock API & Data Generation

13.1 API boundary

`initMockApi()` registers a single router. Endpoints are dispatched to consultation, shop, or health handlers. Failure modes are controlled via `environment.mockFailure`.

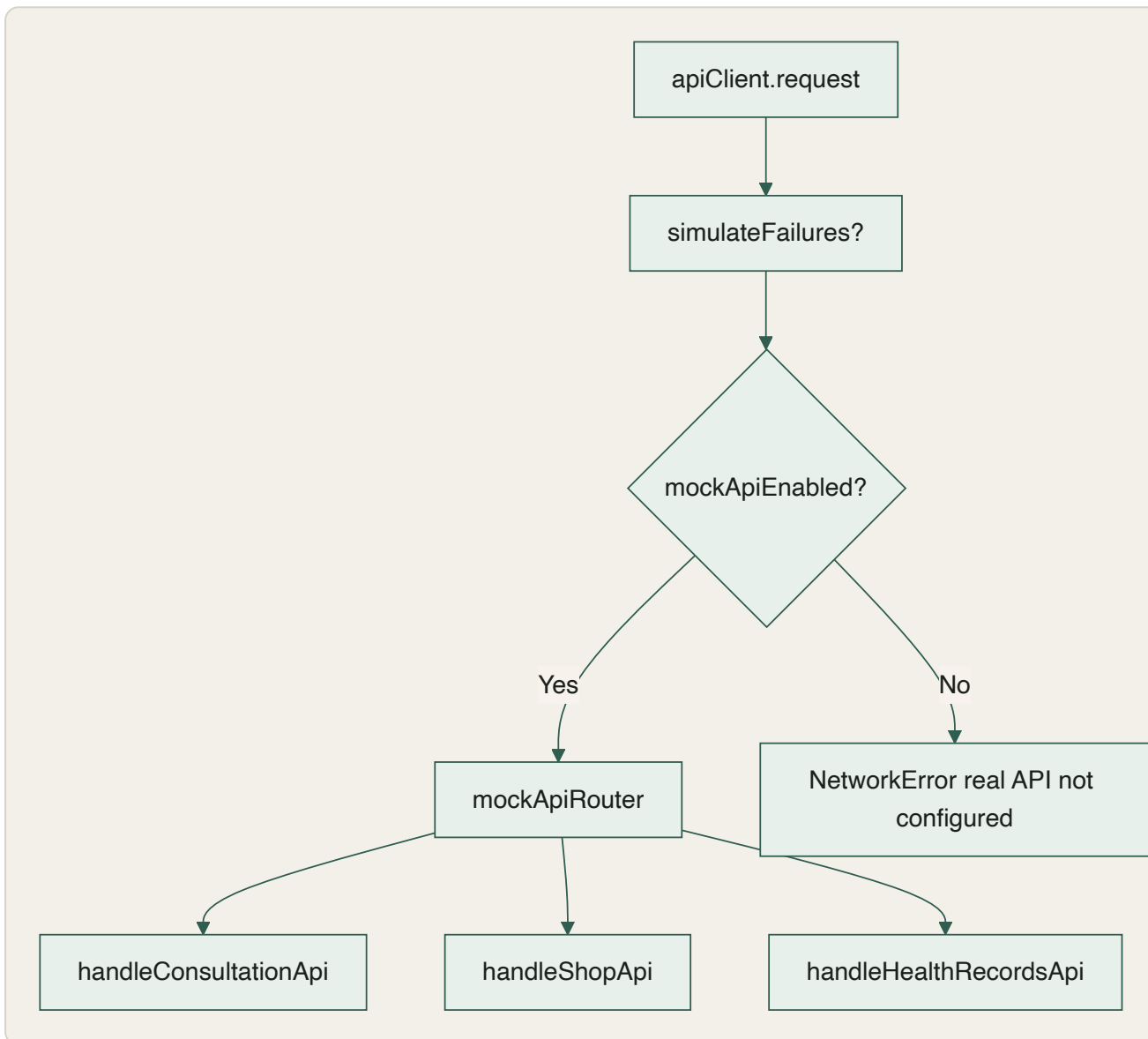


Figure 14 — Mock API routing

13.2 Deterministic generators

Dataset	Count	ID	Seed idea
Doctors	5,000	doc-n	Mulberry32 index * 7919 + 42
Products	20,000	prod-n	index * 3571 + 17
Health records	10,000	hr-n	index * 6271 + 99 ; timeline from Aug 2026 backward
Slots	7 days × 4-8 / doctor	{doc}-slot-...	Derived from doctorId

14. Design System & UX Patterns

- **Tokens:** colors (light/dark), spacing, radius, shadows, typography
- **Primitives:** Text, Button, Input, Card, Badge, ScreenContainer, ScreenHeader, Toast, ErrorBoundary, ListShimmer
- **Offline banner:** ScreenContainer `offline` prop
- **Filter UX:** always-visible type chips; collapsible FilterPanel with Apply / Clear; ActiveFiltersBar
- **Accessibility:** roles, labels, 44pt targets, dual encoding of state (text + badges)

15. Testing Strategy

Layer	Examples
Domain unit	doctorFilters, bookingValidation, productFilters, recordFilters, dateGrouping
Data / pagination	doctorPagination, productPagination, healthRecordPagination, cartStore
Offline	<code>offlineSync.test.ts</code> — queue + reconnect confirm + cached reads
Integration	<code>bookingFlow.e2e.test.ts</code> doctor → slot → book → list
UI	HealthTimelineScreen render smoke test

Run: `npm test`

16. Appendix — Key Paths & Glossary

16.1 Critical files

Area	Path
Bootstrap	<code>src/app/providers/AppProviders.tsx</code>
API	<code>src/core/api/apiClient.ts</code> , <code>mockApiRouter.ts</code>
Cache	<code>src/core/storage/cache.ts</code>
Sync	<code>src/core/sync/syncQueue.ts</code> , <code>syncService.ts</code>
Consultation repo	<code>src/features/consultation/data/consultationRepository.ts</code>
Shop repo / cart	<code>productRepository.ts</code> , <code>cartStore.ts</code>
Health timeline	<code>HealthTimelineScreen.tsx</code>
Architecture notes	<code>ARCHITECTURE_DECISIONS.md</code> , <code>README.md</code>

16.2 Glossary

Term	Meaning
Idempotency key	Client token ensuring one logical booking despite retries
Fresh-or-stale cache	Prefer unexpired entry; else serve expired entry offline
Streaming pagination	Scan indices until page filled; avoid full filtered arrays
Force Offline	Dev override of NetInfo for offline demos
lazyScreen	Code-split stack screen with Suspense fallback

End of document · Amrutam Super App Technical Documentation v1.0 · Generated for assignment review. Flowcharts rendered with Mermaid; implementation source of truth is the repository.